Data Mining and Knowledge Discovery manuscript No.
(will be inserted by the editor)

Mining Regions of Interest from Trajectories: An Optimization Approach

Alexandre Dubray · Guillaume Derval · Siegfried Nijssen · Pierre Schaus

Received: date / Accepted: date

Abstract The number of location-tracking devices is constantly increasing and so does the amount of captured trajectory data. There is hence an emerging need for scalable trajectory mining tools to extract knowledge from mobility data. In this work we focus on the problem of partitioning a space into meaningful regions of interest (ROIs) that are most frequently crossed by trajectories. This problem is of great importance as it constitutes the first step of the possible subsequent analysis of trajectories. Our contribution is that we formulate the task of discovering ROIs as a discrete optimization problem aiming to compress the dense regions from binary density matrix with a given number of simple parameterized shapes, such as rectangles. We give a linear program for solving this optimization problem assuming a given fixed number of ROIs. We then extend the approach to automatically discover the best number of ROIs by relying on the Minimum Description Length Principle. Our experiments on real and synthetic data show that the approach is scalable and able to retrieve ROIs of higher quality than those extracted with the well-known PopularRegion algorithm.

Keywords Data mining · Constrained optimization · Integer Linear Programming · Regions of Interest · Constrained Clustering

A. Dubray
Institute of Information and Communication Technologies, Electronics and Applied Matehmatics (ICTEAM), UCLouvain, Belgium
E-mail: alexandre.dubray@uclouvain.be

G. Derval
Institute of Information and Communication Technologies, Electronics and Applied Matehmatics (ICTEAM), UCLouvain, Belgium
E-mail: guillaume.derval@uclouvain.be

S. Nijssen
Institute of Information and Communication Technologies, Electronics and Applied Matehmatics (ICTEAM), UCLouvain, Belgium
E-mail: siegfried.nijssen@uclouvain.be

P. Schaus
Institute of Information and Communication Technologies, Electronics and Applied Matehmatics (ICTEAM), UCLouvain, Belgium
E-mail: pierre.schaus@uclouvain.be

1 Introduction

An important step in the analysis of trajectory data is the discovery of interpretable regions of interest (ROIs). This task consists in identifying regions frequently crossed by trajectories. This is illustrated in the center of Figure 1 where possible regions of interest are the rectangles A, B, C, D most frequently crossed by some trajectories. These regions of interest can then be used for further analysis of the trajectories.

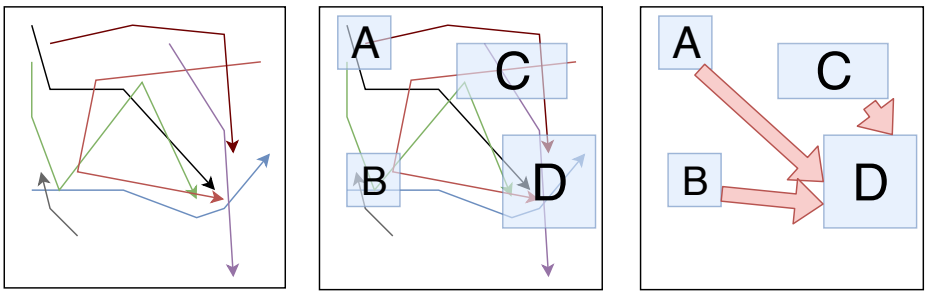


Fig. 1: Following the approach of (Giannotti et al., 2007). Starting from a set of raw trajectories, ROIs are discovered (center part). Finally, all the trajectories are rewritten in terms of these regions and frequent subsequences are reported (assuming a minimum support of 3).

The discovery of ROIs is of practical importance as it can be instrumental for other tasks related to trajectory mining. Three examples of such tasks are the following.

- In (Giannotti et al., 2007), the authors propose to express a trajectory as a sequence of the ROIs it visits. The resulting sequence database can then be analyzed by applying a frequent sequence mining algorithm (Aoga et al., 2016; Agrawal and Srikant, 1995), allowing extracting sequential patterns with a minimum support. This is illustrated at the right of Figure 1. For instance, assuming a minimum support threshold of 3, $B \rightarrow D$ is a subsequence (with possible visits in between) that occurs in at least 3 trajectories. Therefore it is considered to be frequent and informative to report.
- Another possible usage of ROIs is location prediction. This task consists in, given a database of trajectories and the start of a new trajectory of a moving object, predicting what will be the next location of the moving object (Monreale et al., 2009; Ying et al., 2011; Gambs et al., 2012; Mathew et al., 2012). In the area of urban management (Yuan et al., 2011b, 2013), the authors proposed systems to help manage taxis in large cities; these also require algorithms for identifying ROIs.
- Finally, ROIs are also used in the trajectory search problem (Shang et al., 2017) that consists in, given a set of ROIs and a set of trajectories, discovering the trajectories with the highest spatial-density correlation to the query regions.

The most used algorithm for identifying ROIs is the *PopularRegion* algorithm (Giannotti et al., 2007) that is both easy to implement and scalable. This

algorithm extracts non-overlapping rectangular ROIs from a set of trajectories. Intuitively, it first partitions the map by imposing a grid on it, and then for each cell of the grid, counts the number crossing trajectories. A cell is considered dense if its crossing counter is above a given threshold. The cells are greedily expanded/aggregated to form rectangles as long as the average density of the rectangle remains above a given threshold.

The main advantage of the *PopularRegion* algorithm is its scalability and ease of implementation. An important disadvantage is, however, that its output is ill-defined; there is no clear characterization of an objective function that is minimized. Furthermore, as explained in (Gorawski and Jureczek, 2010), it is easy to create examples where the greedy algorithm ends up finding very large ROIs that may hinder the creation of interesting subregions. Finally, another limitation of *PopularRegion* is that it can only generate rectangular regions while the usage of alternative shapes could be more natural for describing some regions. This is illustrated in Figure 2 and 3, which show the initial dense cells as well as the regions discovered by *PopularRegion*, for two different data sets. As can be observed, for both data sets *PopularRegion* identifies regions that cover large part of the city, which is not satisfying. They are a consequence of the fact that only an average density is required for expanding the rectangles.

We introduce a new method for detecting ROIs that addresses the weaknesses of the *PopularRegion* algorithm[1]:

- We state a formal definition for the problem of identifying ROIs. It consists in finding the most parsimonious representation of all the dense cells by using the ROIs.
- For a fixed number k of regions, we show how to discover the optimal regions using an extended formulation of an integer linear programming (ILP) model.
- We relax the constraint of rectangular shapes allowing arbitrary shapes to be chosen from a family of parameterized shapes.
- We show that the number of ROIs can be chosen automatically by using the minimum description length (MDL) principle (Rissanen, 1978).
- A modified integer linear program is introduced for exactly optimizing the MDL criterion and discovering the best set of ROI without a priori specifying their number.
- We evaluate qualitatively the new approach and compare it with *PopularRegion* and DBSCAN (Ester et al., 1996) on both real-life and synthetic trajectory data.

The regions discovered by our method are illustrated in Figures (2c) and (3c). As can be seen, our method finds more fine-grained ROIs and avoids selecting all the isolated cells.

Our paper will start with a discussion of related work in Section 2, followed by our optimization model in Section 3, our approach for identifying the candidate regions that are used in this optimization model in Section 4, experiments in Section 5 and conclusions in Section 6.

[1] The Python code of our method is available at `https://github.com/AlexandreDubray/mining-ROI`

2 State of the art and related work

2.1 The PopularRegion Algorithm

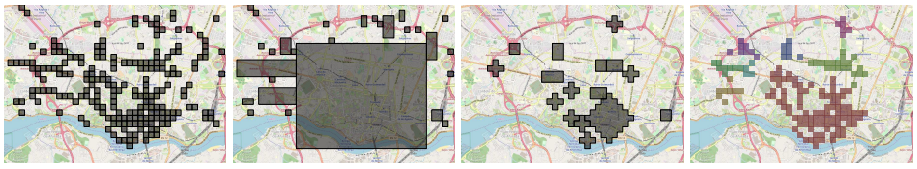
Before introducing the PopularRegion algorithm, we will first introduce notation used in this paper. A trajectory is an ordered sequence of points in $\mathbb{R}^2$ (no temporal information is used in this work). Let $\mathcal{T} = \{T_1, \dots, T_n\}$ be a set of n trajectories, with $T_i = \langle (x_{i1}, y_{i1}), \dots, (x_{im_i}, y_{im_i}) \rangle$. A grid $\mathcal{G}$ of size $M \times N$ is defined on the map with the cell in the i-th row and j-th column denoted by c_{ij} ($1 \leq i \leq M$, $1 \leq j \leq N$). The density of c_{ij} is $\text{density}(c_{ij}) = |\{T \in \mathcal{T} \mid T \cap c_{ij} \neq \emptyset\}|$. Several definitions for the intersection of a cell with a trajectory are possible depending on whether we assume regression points between consecutive entries of the trajectory. A cell c_{ij} is dense with respect to a minimum density threshold θ if $\text{density}(c_{ij}) \geq \theta$. Using the same notation as in (Giannotti et al., 2007), we denote with $\mathcal{G}^* = \{c \in \mathcal{G} \mid \text{density}(c) \geq \theta\}$ the set of all dense cells. *PopularRegion* is an iterative algorithm that first creates a region with a single dense cell and then extends it as much as possible. The extension is made such that the rectangular shape is preserved (e.g. if a region is extended to the left, each cell of the grid that shares its right side with the left side of the region will be merged with it). For an extension in a given direction to be possible, the following conditions must be respected:

- There must be at least one dense cell in the extension.
- There must not be a cell in the extension that is already in another region.
- The resulting new region must have an average density greater or equal to the minimum density threshold θ.

The outline of the algorithm is as follows:

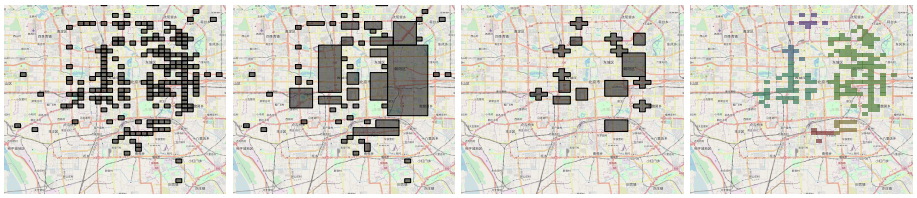
1. Take the cell in $\mathcal{G}^*$ with the highest density that is not already in a ROI. If there is none, return the set of ROIs.
2. Create a ROI with this single cell.
3. While there is a direction in which we can extend the ROI, extend it in the direction that gives the highest average density.
4. Add the ROI to the set of ROIs and go to 1.

However, as pointed out in the introduction, the greedy nature of this algorithm has a number of weaknesses that we aim to address in this work.



(a) Dense cells on a 100×100 grid (b) *PopularRegions* (c) Our method with ratio constraints on the rectangles (d) DBSCAN, each color represents a cluster

Fig. 2: Visualization of the output of the different methods for the Kaggle data set.



(a) Initial set of dense cells on a 200×200 grid (b) *PopularRegions* (c) Our method with ratio constraints on the rectangles (d) DBSCAN, each color represents a cluster

Fig. 3: Visualization of the output of the different methods for the T-Drive data set.

2.2 Other ROI detection algorithms

In (Gorawski and Jureczek, 2010) the authors proposed a modified version of the *PopularRegions* algorithm. They propose to limit the size of the rectangles during the extension process by imposing maximum and minimum size of the rectangles. It thus requires two additional parameters.

In (Belcastro et al., 2018) the authors used geo-tagged social-media data to infer Regions of Interest around predefined points-of-interest (PoI). Their idea is that, by using these metadata (e.g., tag on photos, orientation), they can better infer semantically significant regions. In their work, each data point is associated with a set of keywords (its metadata) and a PoI is also defined by a set of keywords. For a PoI they select the set of data points that share at least one keyword with it. Then they construct the convex hull of this set of points, which they call cp_0. They construct the sequence of polygons $\{cp_1, \ldots, cp_n\}$ such that cp_i is obtained by removing a vertex of cp_{i-1}. They remove the vertex that results in the polygon of the highest density, the density being defined by the number of PoI in the polygon divided by its area. Then they select as ROI cp_{cut} such that, for $\{cp_0, \ldots, cp_{cut-1}\}$ we have cp_i significantly larger than cp_{i+1} and the same condition for $\{cp_{cut+1}, \ldots cp_n\}$.

2.3 Stay Points

ROIs are related to Stay Points (SPs); even though we will focus our attention in this paper on the identification of ROIs, it should be noted that our method can be applied, with minor modifications, on such related problems as well. A Stay Point (SP) is a geographic region where a user stays for a significant amount of time. A Stay Point detection algorithm was first proposed in (Li et al., 2008) to discover regions that have a semantic meaning (e.g. restaurant, mall, etc.). In order to identify a SP in a trajectory sequence $\langle s_1, s_2, \ldots, s_n \rangle$, each data point is considered in order. For each point, the subsequent ones that are close enough (in time and distance) are clustered together with it to form a stay point. Note that with this algorithm, not all data points are transformed into a SP. Indeed, let us consider the case where s_0 is far away from s_1 but the travel time is short. Then the user did not stay long enough in s_0 and s_0 is not considered a stay point.

This approach was extended in (Yuan et al., 2011b, 2013) to cluster the SPs and increase the precision.

The *PopularRegion* algorithm, as well as our algorithm, could be used for identifying stay points as follows: rather than defining the density of a cell as the number of crossing trajectories, it can be defined as the relative fraction of time spent in the cell by a trajectory. Hence, as long as a weight for the cells can be defined, our method will remain applicable.

2.4 Clustering

The task of finding ROIs on the grid shares some similarities with clustering of dense cells. The main difference is that the clusters are generally not constrained in shape (a cluster is an arbitrary collection of cells) and are thus less interpretable than (rectangle) ROIs. DBSCAN (Ester et al., 1996) is one of the most popular density-based clustering algorithms. It doesn't require to specify the number of clusters and is also able to identify outliers. It takes two parameters, a minimum distance ϵ and a minimum number of data points *minPts*. The DBSCAN algorithm identifies core points as the ones with at least *minPts* points within the ϵ distance neighborhood. The connected components of core points on the ϵ neighborhood graph define the clusters. The non-core points are either considered as noise (outliers) or assigned to the closest cluster depending on whether it is further than ϵ distance. Examples of output of DBSCAN are shown in Figures (2d) and (3d). We can observe clusters of various form since they are not constrained by the algorithm. Even though it adds some flexibility to the approach, in some cases it can be less interpretable than an approach with predefined parameterized shapes, like *PopularRegion* or the method proposed in this work.

In (Cai et al., 2014), the authors propose to split the rectangular regions found by *PopularRegion* into connected component of dense cells. They proceed as follows

1. Take the cell in the rectangle with the highest density that is not already in a ROI. If there is none, return the set of ROIs.
2. Create a ROI with this single cell.
3. Take a cell c, in the ROI that has not been extended. If there is none, add the ROI to the set of ROI and go to 1.
4. Add to the ROI every dense cell adjacent to c with lower density and that is inside the same rectangle. Go to 3.

And they do it for every rectangle. This method, as *PopularRegions*, covers every dense cell but it excludes all non-dense cells from the ROIs. The final ROIs are not thus of predefined shapes but are simply sets of connected dense cells. This method is thus not able to filter outliers like DBSCAN and produces ROI difficult to interpret since not constrained in shape.

3 An optimization model for ROIs

The main intuition behind our approach is that we formalize the problem of finding a small number of ROIs as a *discrete constrained optimization problem* that can be solved using *integer linear programming* (Conforti et al., 2010).

We will introduce our approach in two steps. First, we will introduce a formalization of the problem for a fixed number of K ROIs. Subsequently, we will propose a small modification of this approach that allows automatically determining the number of regions K, by using the Minimum Description Length Principle.

3.1 An exact model for encoding the grid

In our basic model, we aim to find K non-overlapping rectangles aligned to the axes of the grid. The model assumes that all the cells covered by the rectangles are dense and the other cells are non-dense. Of course, such a model will make a number of errors: the non-dense cells contained in some rectangle and the dense cells not covered by any rectangle. In the example of Figure 4, the model has two rectangles and four errors: the cells (4,3) and (6,7) are non-dense cells covered by a rectangle, and the cells (6,2) and (7,8) are dense cells not covered by a rectangle.

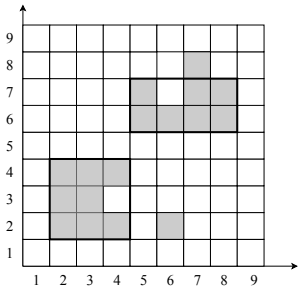


Fig. 4: Model Example

The quality of a model can thus be characterized by its error, i.e. the number of squares wrongly categorized by the selection of rectangles.

Let the set of K rectangles in our model (the regions of interest) be $\mathcal{R} = \{r_1, \ldots, r_K\}$. Each rectangle is defined by its bottom left and top right coordinates (x_k^1, y_k^1), (x_k^2, y_k^2). By abuse of notation, we also denote by r the set of cells covered by some rectangle r and by $\mathcal{R}$ the set of cells covered by all the rectangles in $\mathcal{R}$. The dense cells not covered by any rectangle are denoted as $error^+ = \{c \in \mathcal{G} \mid density(c) \geq \theta \wedge c \notin \mathcal{R}\}$ and the non-dense cells covered by some rectangles are denoted as $error^- = \{c \in \mathcal{G} \mid density(c) < \theta \wedge c \in \mathcal{R}\}$.

In our optimization model, we hence wish to discover the set of rectangles that minimizes the sum of lengths of the error terms: $\arg\min_{\mathcal{R}} |error^+| + |error^-|$

3.2 Basic ILP Formulation

The most basic approach would be to model this problem as an integer linear program by introducing $4K$ integer variables to represent the bottom left and top right corners of each rectangle (two variables per corner). Thus we have, for a rectangle, $r_k = (r_k^1, c_k^1, r_k^2, c_k^2)$ with the pair (r_k^1, c_k^1) representing the bottom left

corner, and (r_k^2, c_k^2) the upper right one. The r variables represent the rows and the c variables the columns of the corners. In order to compute the objective, binary variables $cov_c \in \{0,1\}$ are introduced for each cell c, the variable being equal to 1 if and only if the cell is covered by a rectangle. We model that a rectangle r_k covers a cell c with a variable $cov_c^k \in \{0,1\}$ being equal to 1 if this is the case and 0 otherwise. For all cell $c = (i,j) \in \mathcal{G}$ and rectangles $r_k \in \mathcal{R}$, we have $up_c^k \in \{0,1\}$ that is 1 if $r_k^1 \leq i$ (the cell is upper than the lowest row of the rectangle) and $down_c^k \in \{0,1\}$ that is 1 if $i \leq r_k^2$. We define the corresponding variables on the column, $left_c^k \in \{0,1\}$ that is 1 if $j \leq c_k^2$ and $right_c^k \in \{0,1\}$ that is 1 if $c_k^1 \leq j$. The model is given next for a $R \times C$ grid. In this model, we denote c_{ij} a cell $c \in \mathcal{G}$ that is at the i-th row and j-th column.

$$\text{minimize} \sum_{c \in \mathcal{G}^*} (1 - cov_c) + \sum_{c \in (\mathcal{G} \setminus \mathcal{G}^*)} cov_c \tag{1a}$$

subject to

$$-R(1 - cov_c^k) + r_k^1 \leq i \leq r_k^2 + R(1 - cov_c^k) \qquad \forall c_{ij} \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1b}$$

$$-C(1 - cov_c^k) + c_k^1 \leq j \leq c_k^2 + C(1 - cov_c^k) \qquad \forall c_{ij} \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1c}$$

$$-R \cdot up_c^k < r_k^1 - i \leq R(1 - up_c^k) \qquad \forall c_{ij} \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1d}$$

$$-R \cdot down_c^k < r_k^2 - i \leq R(1 - down_c^k) \qquad \forall c_{ij} \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1e}$$

$$-C \cdot right_c^k < c_k^1 - j \leq C(1 - right_c^k) \qquad \forall c_{ij} \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1f}$$

$$-C \cdot left^k < c_k^2 - j \leq C(1 - left^k) \qquad \forall c_{ij} \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1g}$$

$$up_c^k + down_c^k + left_c^k + right_c^k \leq cov_c^k + 3 \qquad \forall c \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1h}$$

$$up_c^k + down_c^k + left_c^k + right_c^k \geq 4cov_c^k \qquad \forall c \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1i}$$

$$cov_c = \textstyle\sum_{k=1}^{K} cov_c^k \qquad \forall c \in \mathcal{G} \tag{1j}$$

$$cov_c \in \{0,1\}, cov_c^k \in \{0,1\} \qquad \forall c \in \mathcal{G} \tag{1k}$$

$$up_c^k \in \{0,1\}, down_c^k \in \{0,1\} \qquad \forall c \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1l}$$

$$right_c^k \in \{0,1\}, left_c^k \in \{0,1\} \qquad \forall c \in \mathcal{G}, \forall r_k \in \mathcal{R} \tag{1m}$$

Constraints (1b)-(1i) ensure that we have $cov_c^k = 1 \Leftrightarrow c \in r_k$. The constraints (1b)-(1c) impose $cov_c^k = 1 \Rightarrow c \in r_k$ while constraints (1d)-(1i) impose the converse. For the converse, constraints (1d)-(1g) impose the values for the $up_c^k, down_c^k, left_c^k, right_c^k$ variables, while (1h)-(1i) make the link with the cov_c^k variables. The minimization of the objective function (1a) directly translates the minimization of the errors, the first term of the sum represents $|error^+|$ and the second term represents $|error|^-$.

Unfortunately, such an optimization model has several severe limitations, preventing it from being solved efficiently. The first one is that all the rectangles are interchangeable in the model, inducing symmetries known for being extremely difficult to solve (Margot, 2010). The second one is that this model introduces *Big-M constraints* (1b)-(1e). These constraints use a big constant (usually called M) in order to be activated or deactivated depending on the value of a binary variable. These constraints are known for weakening the linear programming bounds. As a consequence, the branch and bound exploration will prune less and more nodes need to be visited.

Therefore, extended formulations are generally preferable (Vanderbeck and Wolsey, 2010) since they can avoid the symmetries in the model and remove the need for big-M constraints resulting in a model that is easier to solve in practice with tighter bounds. Although the extended formulations require a larger number of variables, for our application, the model remains polynomial in the size of the grid for this problem (when this is not the case, one can generate them lazily in a column generation fashion). Such an extended formulation is developed next.

3.3 Extended formulation

The extended formulation relies on a precomputed set of candidates rectangles $\mathcal{S}$ (shapes). It introduces one binary variable x_i for every candidate rectangle $r_i \in \mathcal{S}$. For a $R \times C$ grid, there are less than $R^2C^2 = |\mathcal{G}| \times |\mathcal{G}|$ such rectangles, that is, the total number of possible pairs of coordinates. The improved extended formulation model is as follows:

$$\text{minimize} \sum_{c \in \mathcal{G}^*} (1 - cov_c) + \sum_{c \in (\mathcal{G} \setminus \mathcal{G}^*)} cov_c \tag{2a}$$

subject to

$$\sum_{r_i \in \mathcal{S}} x_i \leq K \tag{2b}$$

$$\sum_{r_i \in \mathcal{S} | c \in r_i} x_i \leq 1 \qquad \forall c \in \mathcal{G} \tag{2c}$$

$$x_i \leq cov_c \qquad \forall c \in \mathcal{G}, \forall r_i \in \mathcal{S} \tag{2d}$$

$$cov_c \leq \sum_{r_i \in \mathcal{S} | c \in r_i} x_i \qquad \forall c \in \mathcal{G} \tag{2e}$$

$$x_i \in \{0, 1\} \qquad \forall r_i \in \mathcal{S} \tag{2f}$$

$$cov_c \in \{0, 1\} \qquad \forall c \in \mathcal{G} \tag{2g}$$

The constraint (2b) limits the number of selected rectangles to K. The constraints (2c) prevent selecting overlapping rectangles. The constraints (2d) and (2e) ensure the variable cov_c of a cell c is 1 if and only if it is covered by a selected rectangle. As can be observed, this model does not suffer from the variable symmetries and does not contain Big-M constraints.

We further improve this model to get rid of the $|G| \times |\mathcal{S}|$ constraints (2d) and (2e) and the binary variables cov_c. This new model relies on the next theorem stating that the value $|error^+| + |error^-|$ can be inferred solely based on the number of dense and non-dense cells covered by the rectangles.

Theorem 1 *By denoting d_k (u_k) the number of dense (non-dense) cells covered by the rectangle r_k, the problem*

$$\arg\min_{\mathcal{R}} |error^+| + |error^-|$$

is equivalent to

$$\arg\min_{\mathcal{R}} \sum_{r_k \in \mathcal{R}} (u_k - d_k).$$

Proof. The term $|error^+|$ can be written as $|\mathcal{G}^*| - \sum_{r_k \in \mathcal{R}} d_k$. The term $|error^-|$ is $\sum_{r_k \in \mathcal{R}} u_k$. □

The extended compact linear program to solve is then the following:

$$\text{minimize} \sum_{r_i \in \mathcal{S}} x_i \cdot (u_i - d_i) \tag{3a}$$

subject to

$$\sum_{r_i \in \mathcal{S}} x_i \leq K \tag{3b}$$

$$\sum_{r_i \in \mathcal{S} | c \in r_i} x_i \leq 1 \qquad \forall c \in \mathcal{G} \tag{3c}$$

$$x_i \in \{0, 1\} \quad \forall r_i \in \mathcal{S} \tag{3d}$$

with equation (3b) limiting the number of regions and equation (3c) enforcing non-overlap between the regions. The problem being solved by this linear program is actually an instance of the Maximum Weighted Independent Set problem with an additional cardinality constraint (Kalra et al., 2017).

3.4 ROIs of arbitrary shape

It is straightforward to modify the extended formulation to deal also with non-rectangular shapes. Any shape that covers a set of cells can be included in the candidate set $\mathcal{S}$ of the model. For instance, circular regions are natural candidates for ROIs. A circular region $c = (row, col, radius)$ is defined by its center and its radius. It covers the cells $c = \{c_{ij} \in \mathcal{G} \mid |i - row| + |j - col| \leq radius\}$ (assuming Manhattan distance). For a circular region c_k, we can also define d_k and u_k as the set of dense/non-dense cells covered by the regions of interest.

Not all possible candidate regions need to be considered in $\mathcal{S}$. It is clear that we can discard all regions that contain more non-dense cells than dense ones. Moreover, in order to have more homogeneous ROIs, some ratio constraints can be imposed on the candidate rectangle ROIs. For instance, we may a priori filter-out too flat rectangles (ratio constraint). We further discuss the choice of candidates in Section 4.

3.5 A parameter-free approach

Fixing the limit K for the maximum number of ROIs can in some cases be an arbitrary decision. We can use the *Minimum Description Length* (MDL) principle (Rissanen, 1978) to determine the size of a model in a principled manner. It trades off the description length of the data given the model, and the description length of the model itself. More precisely, let us assume that we have a set of models (hypothesis) $\mathcal{H}$. The description *length* of the model $L(D)$ is the number of bits needed to encode the model; the description length of the data $L(D \mid H)$ is the number of bits needed to encode the data given the model H. The MDL principle tells us to prefer the model that minimizes $L(D, H) = L(H) + L(D \mid H)$.

The model described in the previous sections is composed of a choice of multiple ROIs, indicating where the cells must be dense, along with errors of the model, that is, coordinates of cells which are included in a selected ROI but are non-dense, and dense cells outside the selected ROIs. Each of these needs a different number of values to be encoded:

- 4 values per rectangle (top-left and bottom-right corners coordinates)
- 3 values per circle (center coordinate and radius)
- (other types of ROIs are possible)
- 2 values per wrongly classified cells (coordinates of the cell)

A fixed number of bits are used for every value.

The length of a model $\mathcal{S}$ (selected ROIs) and of a given grid $\mathcal{G}$ given $\mathcal{S}$ is then the following:

$$L(\mathcal{S}) = \sum_{r_i \in \mathcal{S}} size(r_i) \qquad L(\mathcal{G} \mid \mathcal{S}) = 2 \cdot (D + \sum_{r_i \in \mathcal{S}} (u_i - d_i)),$$

where $size(r_i)$ is the number of values required to encode ROI i (3 if i is a circle and 4 if it is a rectangle, for example) and D is the number of dense cells in the grid. The value $D + \sum_{r_i \in \mathcal{S}} (u_i - d_i)$ counts the number of errors made by the model and the factor 2 accounts for encoding the two coordinates of each exception cell.

We can use the MDL criterion to discover the regions of interest without fixing their number in advance. To find regions that minimize the MDL criterion, we solve the following ILP model:

$$\text{minimize} \sum_{r_i \in \mathcal{S}} x_i \cdot (2(u_i - d_i) + size(r_i)) \tag{4a}$$

subject to

$$\sum_{r_i \in \mathcal{S} \mid c \in r_i} x_i \leq 1 \qquad \forall c \in \mathcal{G} \tag{4b}$$

$$x_i \in \{0, 1\} \quad \forall r_i \in \mathcal{S} \tag{4c}$$

Note that we do not exactly minimize $L(\mathcal{G}, \mathcal{S})$. Indeed we removed the constant $2 \cdot D$ as it does not impact the optimization.

4 Generation of candidate ROIs

The number of possible rectangles is polynomial in the size of the grid; more precisely, for a $M \times N$ grid, there are less than $M^2 N^2$ possible rectangles (all the coordinates $(x^1, y^1), (x^2, y^2)$). There is also a polynomial number of circles. Naively enumerating all the possible ROIs may result in an overly large number of ROI candidates and each requires the introduction of one variable for the ILP model. Fortunately, one can avoid generating rectangles with fewer dense cells than non-dense cells. This will be the case for most of them in practice. Also, a rectangle with one or more contiguous line/column containing more non-dense than the number dense cells plus two can also be filtered out. The reason is that it becomes more interesting in terms of description length to split this rectangle in two and record the exception cells on that line.

As an example of the effectiveness of the filtering, Table 1 shows, for multiple configurations on the Kaggle data set, the total number of distinct rectangles before and after the filtering.

Min. density threshold	Grid Size	# candidates	# remaining candidates
2	100	25 502 500	17 218
	150	128 255 625	7 703
	200	404 010 000	3 330
5	100	25 502 500	2 523
	150	128 255 625	1 255
	200	404 010 000	448

Table 1: Impact of the filtering on the set of possible candidates

5 Results and comparison

Our experiments compare the *PopularRegion* algorithm with our new approach on both real and synthetic data. Although clustering techniques are not producing geometrical shapes ROIs and can thus not directly be compared in terms of interpretability and expressivity, we also include DBSCAN in our comparisons, assuming that the clusters are the model.

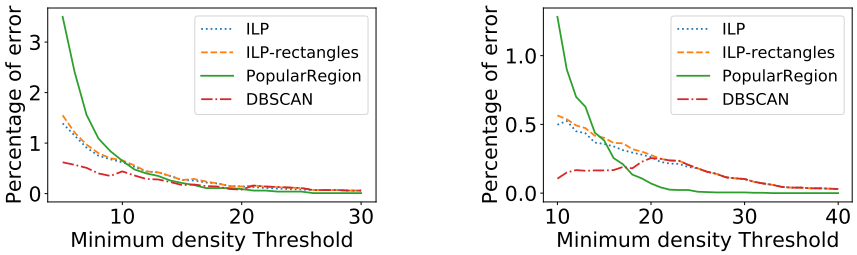
For the rest of this section, we denote by *ILP* our full model (i.e., equations (4a)-(4b)) that includes rectangular and circular ROIs while *ILP-rectangles* denote a restricted model containing only rectangular ROIs. In this section we will address the following questions: i) How well does our method perform compared to *PopularRegion* and DBSCAN? ii) How efficient is our approach and what is the computation bottleneck ? iii) Is our method robust to noise in data? We do not compare with the approach proposed in Cai et al. (2014) since it will always have an MDL error equal to twice the number of dense cells in the grid (two coordinates per dense cell). Indeed this method does not attempt to generalize the regions or exclude outliers (like DBSCAN). Instead if finds an exact partition of the dense cells.

5.1 Performances with respect to the MDL criterion

We first describe an experiment performed on two real-world data sets. The first comes from the taxi destination prediction challenge that was organized by the 2015 ECML/PKDD conference and proposed as a Kaggle competition. This data set contains more than 1.6 million trajectories from taxis of the city of Porto[2]. The second data set is the T-Drive data set from Microsoft and contains GPS traces from taxis of Beijing (Yuan et al., 2011a, 2010). For the Kaggle data set, the density threshold will be expressed as a percentage of the total number of trajectories and we used a 100×100 grid. For the T-Drive data set, we used a 200×200 grid and, since we do not have separate trajectories, the density threshold will be a percentage of the maximum density in the grid. For DBSCAN we fixed the parameters $\epsilon = 2$ and $minPts = 6$. These values were chosen experimentally to form clusters with a number of cells similar in size to the rectangle ROIs found by our method.

[2] The data set can be downloaded at this link `https://www.kaggle.com/crailtap/taxi-trajectory/home`. We filtered out incomplete trajectories and the few trajectories that went too far away from Porto.

Figure 5 shows the error rate, that is, the percentage of cells wrongly classified by the regions $(|error^+| + |error^-|)/(M \cdot N)$ in function of the minimum density threshold. As explained before and illustrated in Figure 2b and 3b, for low-density thresholds, *PopularRegion* tends to create large regions, which results in a high error rate since it covers many non-dense cells. As the cluster of dense cells found by DBSCAN constitutes its model, the only error rate is induced by the dense cells considered as noise. The error rate of DBSCAN should thus be considered as a very optimistic baseline for comparison. Our method discovers regions that generalize the initial distribution of the dense cells well, and allows some non-dense cells in the ROIs. The error rate is generally between the one of *PopularRegion* and DBSCAN. When the minimum density threshold increases, DBSCAN and our approach perform slightly worse than *PopularRegion*. The reason is that *PopularRegion* will over fit perfectly the isolated dense cells by creating one region for each, which is obviously not the expected behavior of an algorithm for detecting ROIs. As expected, the addition of circular shapes permits decreasing slightly the error rate over the rectangle model since it augments the expressivity of the ROIs.



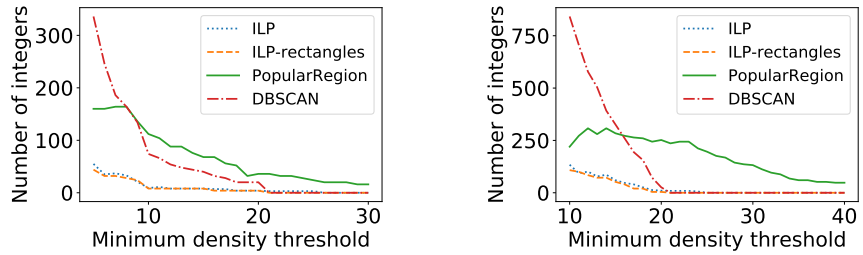
(a) Kaggle data set

(b) T-drive data set

Fig. 5: Percentage of error in function of the minimum density threshold

Figure 6 shows the number of integers needed to encode the ROIs (i.e. the first part of the MDL criterion, excluding the values needed to encode errors). Our method always gives a smaller value, with and without circular regions. It can be seen that for a high density threshold, the number of ROIs tends to zero as it is more advantageous to store the exceptions directly rather than using ROIs. For DBSCAN we assume, two integers are needed per dense cells in the clusters. When the minimum density threshold becomes larger, the dense cells become sparse over the map and DBSCAN considers them as noise without identifying any cluster.

Figures 5 and 6 show that we outperform *PopularRegion* on low threshold values by having a smaller error rate and nonetheless using fewer ROIs. On higher value, our methods maintain a similar error rate as *PopularRegion* while using at least four times less ROIs. Compared to DBSCAN, we have a higher error rate due to the inclusion of non-dense cells in the ROIs, but our ROIs are more interpretable as they require less integers for their encoding.



(a) Kaggle data set

(b) T-drive data set

Fig. 6: Number of integers needed to encode the ROI positions, in function of the minimum density threshold.

5.2 Execution times

Table 2 shows the run time of the methods for two minimum density thresholds and three grid sizes for each data set. For the ILP model, we show the time needed to solve the optimization problem defined in Equations (4a)–(4b). The table also shows the total number of dense cells in the grid as well as the number of candidate shapes.

Minimum density threshold	2%			5%		
Grid side size	100	150	200	100	150	200
Number of dense cells ($\|\mathcal{G}^*\|$)	571	597	537	230	178	137
Number of ILP candidates	23 814	7 779	3 399	2 880	1 232	434
ILP optimization time (s)	4.328	0.464	0.109	0.113	0.044	0.029
PopularRegion run time (s)	0.003	0.005	0.006	0.002	0.003	0.004
DBSCAN run time (s)	0.003	0.003	0.004	0.002	0.002	0.003

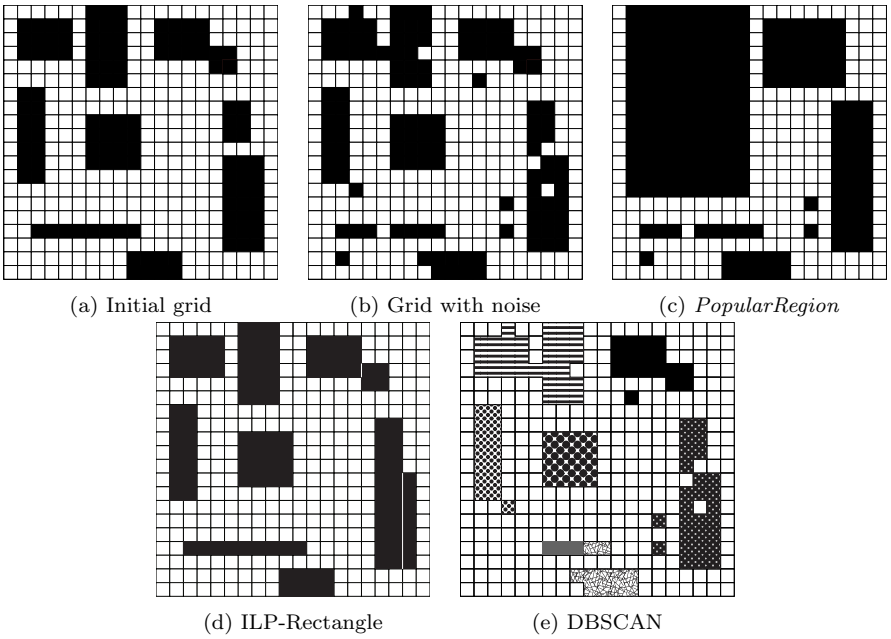
Table 2: Run time of the methods for different grid sizes and minimum density thresholds for the Kaggle data set.

As can be observed, the resolution time of the ILP-based approach is mostly determined by the number of candidate shapes as these correspond to the number of variables in the model. Working on 200×200 grid is already very fine-grained as it corresponds to squares less than $25m^2$. Despite requiring more time than the greedy DBSCAN and *PopularRegion* algorithms, our approach can be considered as practical for identifying ROIs from standard GPS data set.

5.3 Robustness to noise

We use a synthetic data set with known regions of interest to measure the robustness to noise. We start from a 20×20 grid with predefined regions of interest shown

in Figure 7a. We then flip each cell with a probability p to simulate noise. After flipping, assuming a minimum density threshold of 5%, each dense cell receives a random value between 5 and 30 and each non-dense cell one between 0 and 4. We do not go beyond 30 because such values are unlikely in real data sets while being strong outliers for the *PopularRegion* algorithm. An example of input for $p = 0.03$ is shown in Figure 7b. For this particular input, the solution discovered by our algorithm is shown in Figure 7d, a solution produced by *PopularRegion* is shown in Figure 7c and the clusters of DBSCAN are shown in Figure 7e. Our method is able to find (mostly) all the original ROIs in this particular example.



(a) Initial grid (b) Grid with noise (c) *PopularRegion*

(d) ILP-Rectangle (e) DBSCAN

Fig. 7: Example of synthetic data

To compare the output ($\mathcal{R}$) of the methods, we use the following metrics: the recall $|\mathcal{R} \cap \mathcal{G}^*|/|\mathcal{G}^*|$, the precision $|\mathcal{R} \cap \mathcal{G}^*|/|\mathcal{R}|$ and the F1-measure $(2 \cdot precision \cdot recall)/(precision + recall)$. Figure 8 shows how these metrics evolve with the level of noise.

PopularRegion obtains a recall that is constantly close to 1.0 since by construction, all the input dense cells are covered. With noise, the dense cells in the input matrix will tend to be more uniformly distributed and thus *PopularRegion* will cover almost all the matrix. Without noise, the precision is initially good for *PopularRegion*, but as soon as some noise is added, the precision quickly drops, because the ROIs are too large and include many non-dense cells. The precision converges towards 0.3, which is approximately the initial percentage of dense cells in the grid ($119/400 = 0.2975$).

The behavior is different for both DBSCAN and our method. The recall and precision decrease more linearly with the noise. When the level of noise is low,

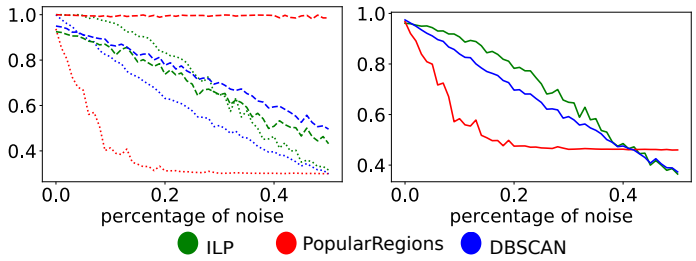


Fig. 8: Recall (dashed line), precision (dotted line) and F1 (continuous line) w.r.t. the original data (without added noise) in function of the added noise, on the synthetic data set

the non-dense cells that are flipped are isolated and our algorithm will not try to cover them as the cost is higher than treating them as exception. Similarly, for DBSCAN, the isolated dense cells will be considered as noise.

When the probability of flipping cells is $p = 0.5$, both methods reaches the same precision value as *PopularRegions*. Indeed, for this probability, the matrix becomes completely random. Thus, for each cell covered by a region, the probability that this cell is dense in the initial matrix is $|\mathcal{G}^*|/400 = 0.2975$. These combined effects make our method perform betters than both *PopularRegion* and DBSCAN when looking at the F1-measure.

6 Conclusion

Following the approach proposed in (Giannotti et al., 2007), we have proposed a new method for discovering non-overlapping regions of interest in a trajectory data set. The new method is solving a well-defined problem attempting to minimize the error rate for predicting the dense/non-dense cell status on the grid. We have introduced an integer linear program for discovering the K best regions and have shown that the approach can be extended to discover the best number of regions based on the minimum description length principle. Our experiments on real and artificial data have demonstrated that for lower threshold values on the density of the cells, the error is reduced as compared to the PopularRegion algorithm (Giannotti et al., 2007). The introduced method was shown experimentally to be efficient whenever the majority of the cells are non-dense.

References

Agrawal R, Srikant R (1995) Mining sequential patterns. In: IEEE International Conference on Data Engineering (ICDE), vol 95, pp 3–14

Aoga JO, Guns T, Schaus P (2016) An efficient algorithm for mining frequent sequence with constraint programming. In: Joint European Conference on Machine Learning and Knowledge Discovery in Databases, Springer, pp 315–330

Belcastro L, Marozzo F, Talia D, Trunfio P (2018) G-roi: Automatic region-of-interest detection driven by geotagged social media data. ACM Transactions on Knowledge Discovery from Data (TKDD) 12(3):27

Cai G, Hio C, Bermingham L, Lee K, Lee I (2014) Sequential pattern mining of geo-tagged photos with an arbitrary regions-of-interest detection method. Expert Systems with Applications 41(7):3514–3526

Conforti M, Cornuéjols G, Zambelli G (2010) Extended formulations in combinatorial optimization. 4OR 8(1):1–48

Ester M, Kriegel HP, Sander J, Xu X, et al. (1996) A density-based algorithm for discovering clusters in large spatial databases with noise. In: Kdd, vol 96, pp 226–231

Gambs S, Killijian MO, del Prado Cortez MN (2012) Next place prediction using mobility markov chains. In: Proceedings of the First Workshop on Measurement, Privacy, and Mobility, ACM, p 3

Giannotti F, Nanni M, Pinelli F, Pedreschi D (2007) Trajectory pattern mining. In: Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining, ACM, pp 330–339

Gorawski M, Jureczek P (2010) Regions of interest in trajectory data warehouse. In: Asian Conference on Intelligent Information and Database Systems, Springer, pp 74–81

Kalra T, Mathew R, Pal SP, Pandey V (2017) Maximum weighted independent sets with a budget. In: Gaur D, Narayanaswamy N (eds) Algorithms and Discrete Applied Mathematics, Springer International Publishing, Cham, pp 254–266

Li Q, Zheng Y, Xie X, Chen Y, Liu W, Ma WY (2008) Mining user similarity based on location history. In: Proceedings of the 16th ACM SIGSPATIAL international conference on Advances in geographic information systems, ACM, p 34

Margot F (2010) Symmetry in integer linear programming. In: 50 Years of Integer Programming 1958-2008, Springer, pp 647–686

Mathew W, Raposo R, Martins B (2012) Predicting future locations with hidden markov models. In: Proceedings of the 2012 ACM conference on ubiquitous computing, ACM, pp 911–918

Monreale A, Pinelli F, Trasarti R, Giannotti F (2009) Wherenext: a location predictor on trajectory pattern mining. In: Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining, ACM, pp 637–646

Rissanen J (1978) Modeling by shortest data description. Automatica 14(5):465–471

Shang S, Chen L, Jensen CS, Wen JR, Kalnis P (2017) Searching trajectories by regions of interest. IEEE Transactions on Knowledge and Data Engineering 29(7):1549–1562

Vanderbeck F, Wolsey LA (2010) Reformulation and decomposition of integer programs. In: 50 Years of Integer Programming 1958-2008, Springer, pp 431–502

Ying JJC, Lee WC, Weng TC, Tseng VS (2011) Semantic trajectory mining for location prediction. In: Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, ACM, pp 34–43

Yuan J, Zheng Y, Zhang C, Xie W, Xie X, Sun G, Huang Y (2010) T-drive: driving directions based on taxi trajectories. In: Proceedings of the 18th SIGSPATIAL International conference on advances in geographic information systems, ACM,

pp 99–108
Yuan J, Zheng Y, Xie X, Sun G (2011a) Driving with knowledge from the physical world. In: Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining, ACM, pp 316–324
Yuan J, Zheng Y, Zhang L, Xie X, Sun G (2011b) Where to find my next passenger. In: Proceedings of the 13th international conference on Ubiquitous computing, ACM, pp 109–118
Yuan NJ, Zheng Y, Zhang L, Xie X (2013) T-finder: A recommender system for finding passengers and vacant taxis. IEEE Transactions on knowledge and data engineering 25(10):2390–2403